

# Final Report

## Dam Control System Simulator

|      |                  |
|------|------------------|
| 과 목  | 임베디드시스템S/W설계     |
| 담당교수 | 김태석 교수님          |
| 학 과  | 컴퓨터정보공학부         |
| 학 번  | 2020202090       |
| 성 명  | 최민석              |
| 날 짜  | 2025. 06. 01 (일) |



# Tables of Contents

## 1. Abstract

## 2. Introduction

- 프로젝트 개요
- 프로젝트 구조
- Task 정의
- Task 간 Semaphore와 Message queue 활용
- 제안서 대비 변경사항
- 동작 예시

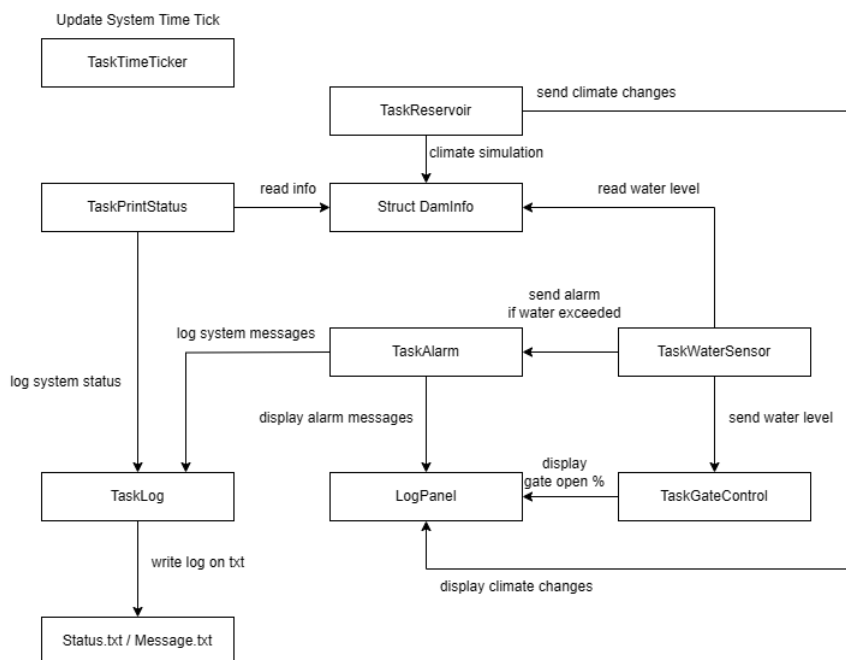
## Abstract

| Kernel Function 개수 | 사용한 Task 개수 | Semaphore 개수 | Message Queue 개수 |
|--------------------|-------------|--------------|------------------|
| 13                 | 7           | 1            | 1                |

본 프로젝트는 RTOS인 uC/OS-II로 구현한 댐 제어 시스템 시뮬레이터이다. 총 7개의 Task와 13개의 Kernel Function, 1개의 Semaphore와 Message queue를 사용하여 동시성과 자원 보호를 처리하였다. 각 Task는 저수지 시뮬레이션, 수위 감지, 수문 제어, 알람 처리, 로그 기록, 상태 출력, 가상 시간 관리 등의 고유 기능을 수행하며, 공유 구조체(DamInfo)를 중심으로 상호작용한다. 특히 Semaphore를 통한 데이터 보호와 Message Queue를 통한 비동기 이벤트 전달 기법을 통해 안정적인 시스템 동작을 구현하였다. 본 시뮬레이터는 설정 파일(config.txt)을 기반으로 초기 환경을 구성하고, 콘솔 인터페이스 및 로그 파일(status.txt, message.txt)을 통해 사용자에게 실시간 정보 및 기록을 제공한다.

## Introduction

### 1. 프로젝트 개요 및 구조

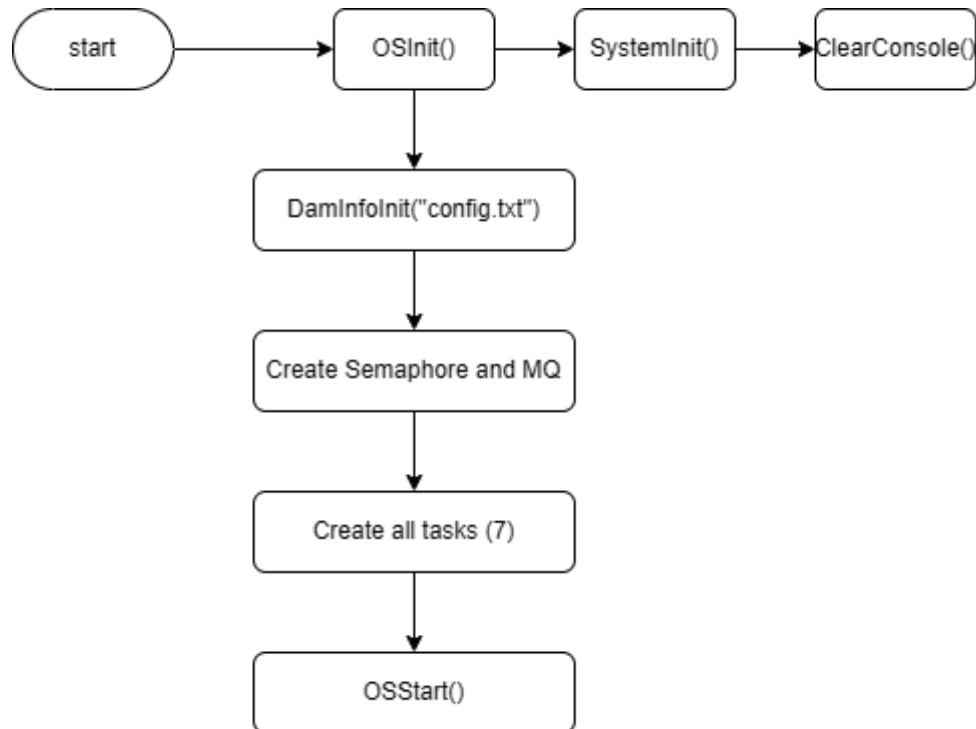


위 다이어그램은 시스템을 구성하는 핵심 요소인 주요 Task들과 구성 요소, 구조체 간 관계를 나타낸 것이다. 각 Task는 DAM\_SIM 내에서 독립적인 역할을 수행하며, 공유 구조체인 DamInfo를 중심으로 동기화 및 데이터 흐름이 이루어진다. 시스템의 기본적인 흐름은 TaskReservoir에서 저수지와 기후 환경을 관리하여 DamInfo에 Water Level을 업데이트하고, 댐 제어 시스템은 DamInfo로부터 수위 및 제어 설정을 읽고, 이를 바탕으로 자동으로 댐의 상태를 제어할 수 있도록 구성하였다. 공유 구조체는 Semaphore를 통해 관리되며, TaskAlarm은 다른 Task로부터 경고 메시지를 Message Queue를 통해 수신하여 알람을 처리하도록 구현되었다.

## 2. 주요 Task 및 플로우 차트

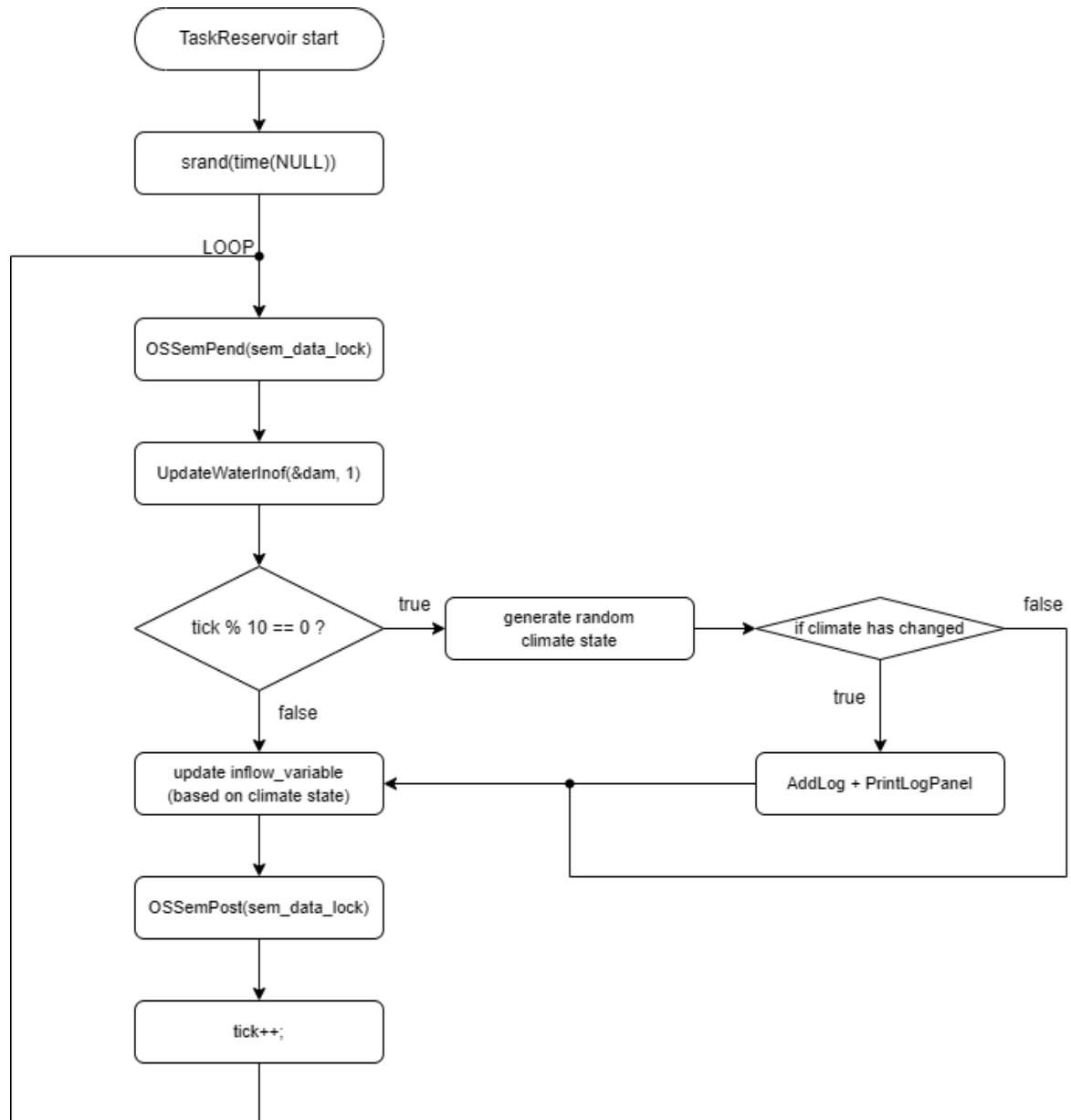
다음은 시스템 다이어그램을 기반으로 main 함수와 각 Task의 동작 방식을 나타낸 플로우 차트이다.

### - main.c



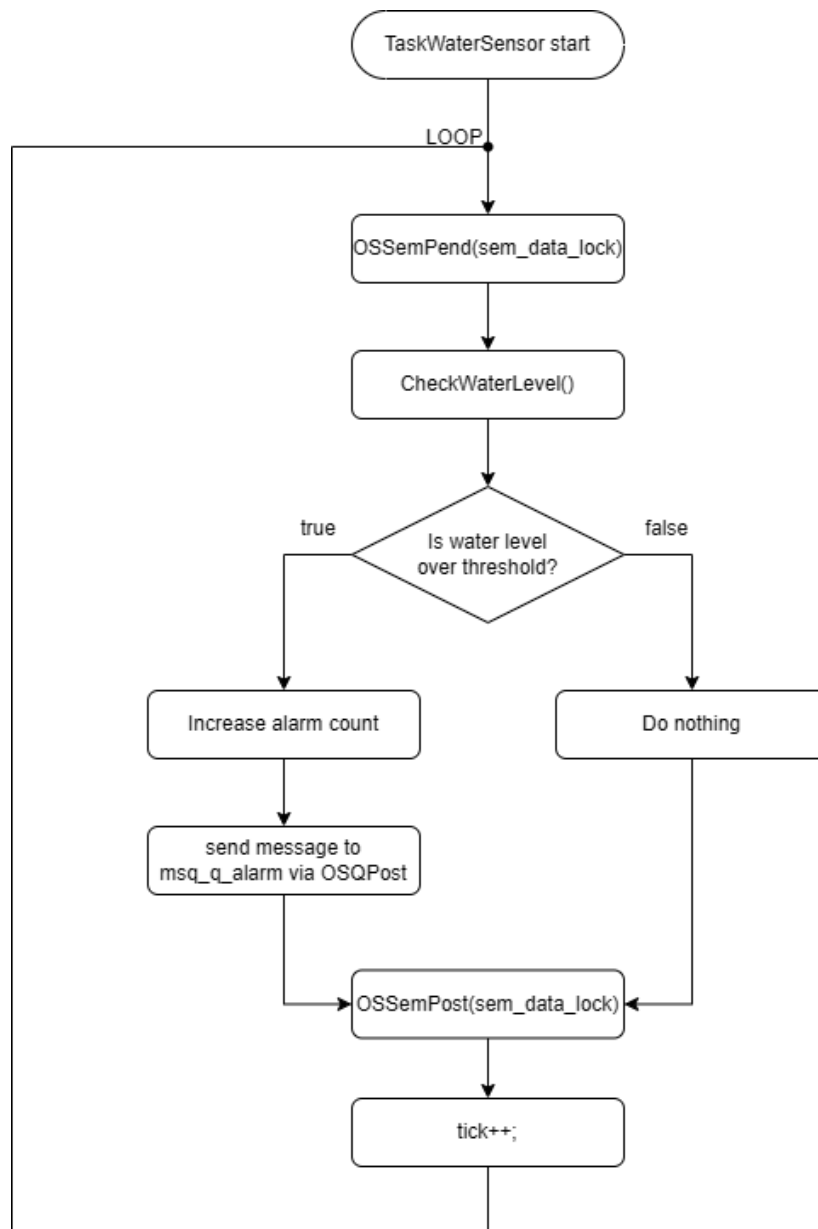
main.c의 주요 동작은 config.txt 파일에서 설정 값을 읽어와 시스템을 초기화한 뒤 각 Task를 생성하고 시스템을 가동하는 것이다. Semaphore와 Message queue를 생성 및 초기화하는 과정도 포함한다.

## - TaskReservoir



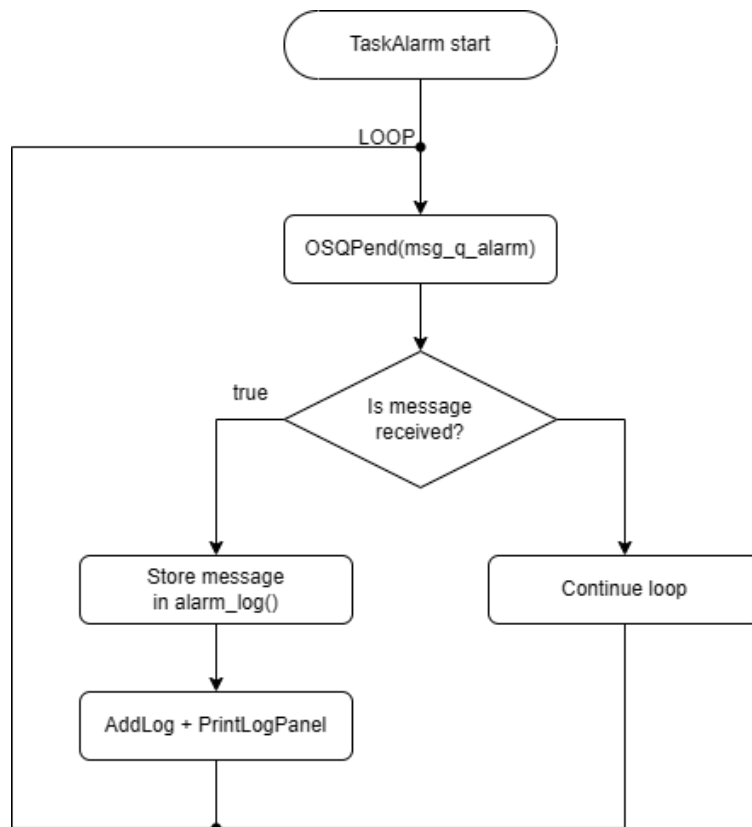
TaskReservoir의 주요 동작은 DamInfo의 inflow 값을 갱신하고, 10초마다 기후를 변경한다. 공유 자원인 DamInfo에 접근하기 전 Semaphore를 획득한 뒤 값을 변경하고 토큰을 해제한다. 기후가 변경될 경우 로그를 남기고 콘솔 로그 패널에도 메시지를 출력한다.

## - TaskWaterSensor



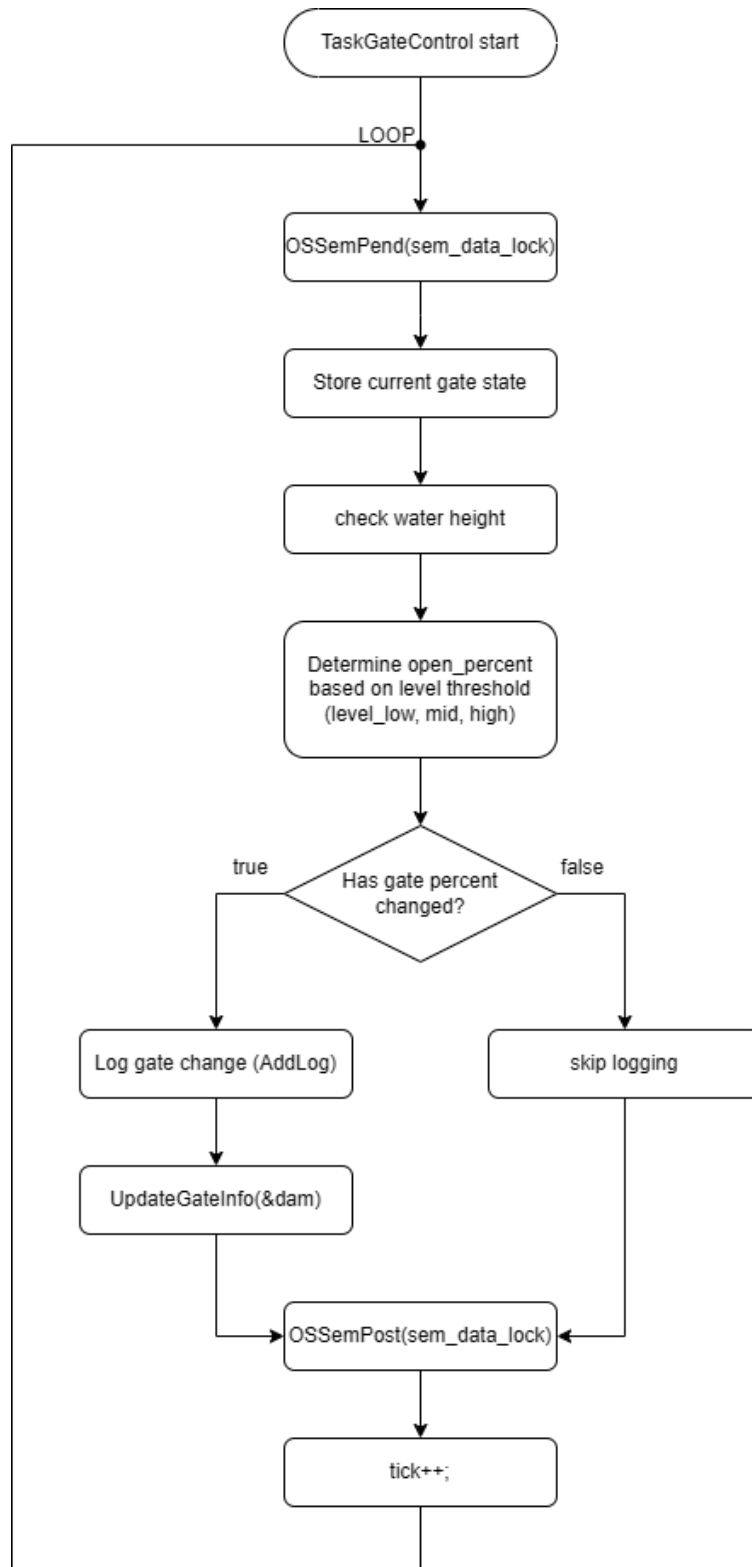
TaskWaterSensor의 주요 동작은 CheckWaterLevel() 함수를 호출하여 DamInfo를 읽고 현재 수위가 알람 임계치를 초과했는지 검사하는 것이다. 수위가 초과된 경우 알람 카운트를 1 증가시키고 Water Level Exceeded!라는 문자열을 msg\_q\_alarm에 전송하여 TaskAlarm이 메시지를 수신할 수 있도록 한다. 이후 Semaphore를 해제하고 1초간 대기한 후 루프를 반복한다.

## - TaskAlarm



TaskAlarm의 주요 동작은 시스템 내에서 발생하는 경고 메시지를 msg\_q\_alarm으로부터 수신하여 처리하는 역할이다. OSQPend()를 호출하여 메시지 큐에서 알람 메시지를 수신 대기한다. 메시지가 수신되면 내부 순환 버퍼에 저장하고, AddLog()를 호출하여 로그에 기록한다. 그리고 PrintLogPanel()를 호출하여 콘솔 우측에 로그 메시지 패널을 업데이트한다. 이후 다시 루프로 되돌아가 메시지 수신을 대기한다.

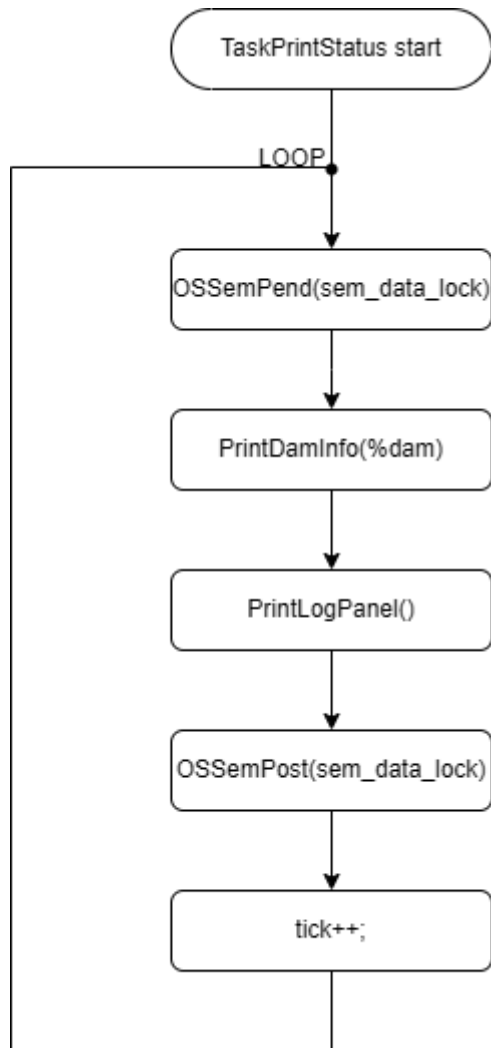
## - TaskGateControl



TaskGateControl의 주요 역할은 댐 수위에 따라 수문 개방률을 자동으로 조절하는 것이다. 시스템은 댐의 수위 정보를 읽고 자동으로 수문 개폐율을 조절하고 발전량을 계산한다. 방출량과 발전량은 수문 개방률에 따른 선형 보간 방법으로 간단하게 구현하였다. 수문 개폐율이 변경되었을 경우 로그를 남기고 콘솔 로그 메시지 패널을 갱신한다. 이후 다시 루프로 되돌아가 위 과정을 반복한다.

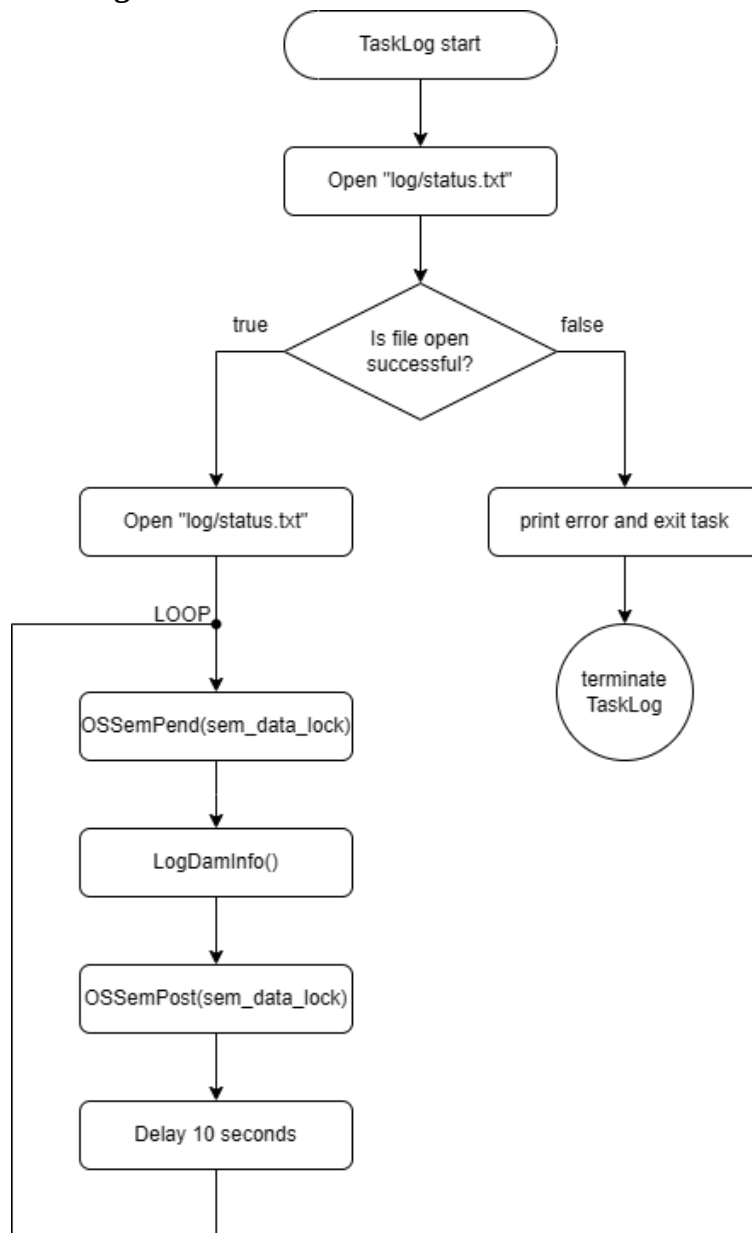


- TaskPrintStatus



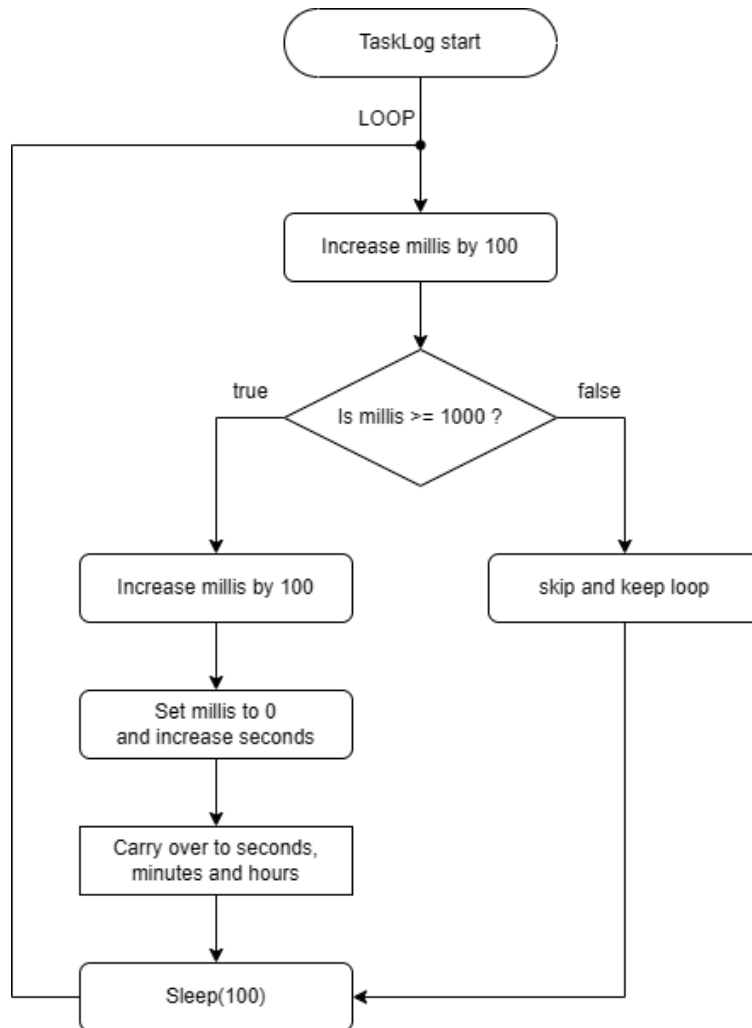
TaskPrintStatus는 콘솔 상에 실시간으로 댐의 상태 정보를 출력한다. 시스템 모니터링 목적으로 사용자 인터페이스 역할을 한다. 먼저 Semaphore를 통해 DamInfo에 대한 접근 권한을 얻고 PrintDamInfo()를 호출하여 현재 수위, 유입량, 수문 개방률, 발전량 등 현재 댐의 상태를 콘솔에 출력한다. 이후 PrintLogPanel()을 호출하여 최근 로그 메시지 목록을 콘솔 우측에 출력하고, Semaphore를 해제한다. 이 과정을 1초마다 수행하여 댐의 상태를 조회할 수 있도록 한다.

## - TaskLog



TaskLog는 댐의 상태를 주기적으로 텍스트 파일에 기록한다. 먼저 log 디렉토리 내 status.txt 파일을 append 모드로 연다. 파일 열기에 실패하면 에러 메시지를 출력하고 Task를 종료한다. 성공적으로 파일이 열리면, 10초 주기로 무한 루프를 돌며 댐의 상태를 기록한다. DamInfo를 읽기 위해 semaphore를 획득하고 LogDamInfo() 함수를 호출하여 댐의 상태를 기록한 후 semaphore를 해제한다. 10초 주기로 루프를 돌며 시스템의 상태를 기록한다.

- TaskTimeTicker



TaskTimeTicker는 시뮬레이션의 가상 시간 흐름을 관리하고 표시한다. 100ms마다 timer.millis 값을 100만큼 증가시키고 이를 표기한다. 다른 Task와 독립적으로 동작하기 때문에 다른 Task가 처리되는 중에도 시스템의 경과 시간을 알 수 있다.

### 3. Task 간 Semaphore와 Message queue 활용

본 시스템에선 Task 간 안정적인 자원 접근과 이벤트 기반 통신을 위해 uC/OS-II에서 제공하는 Semaphore와 Message queue를 활용하였다

#### - Semaphore (sem\_data\_lock)

sem\_data\_lock은 DamInfo의 정보에 대한 공유 자원 보호를 위해 사용된다. 대부분의 Task는 DamInfo 내부의 수위, 개방률, 기후 상태 등을 읽거나 수정하기 때문에 경쟁 조건을 방지하고 데이터 일관성을 보장하기 위해 이 Semaphore를 반드시 획득하고, 작업을 수행한 후 해제하도록 구성하였다.

- TaskReservoir에선 inflow와 climate 상태를 갱신할 때 사용
- TaskWaterSensor에선 수위 감지 및 알람 조건 판단을 위해 사용
- TaskGateControl에선 수문 개방률 계산 및 설정을 위해 사용
- TaskPrintStatus에선 dam 상태 콘솔 출력을 위해 사용
- TaskLog에선 dam 상태 파일 기록을 위해 사용

#### - Message queue (msg\_q\_alarm)

msg\_q\_alarm은 Task 간 비동기 이벤트 전달을 위해 사용된다. 수위 초과 상황 등에서 경고 메시지를 TaskAlarm으로 전달하기 위한 알람 전용 Message queue로서 동작한다. 이를 통해 Task를 직접 호출하지 않아도 알람 메시지를 전달할 수 있어, 우선순위에 따른 Task 실행을 대기하는 것 보다 효율적이라고 생각해서 TaskAlarm에 Message queue를 적용하였다.

- TaskWaterSensor에선 임계 수위를 초과한 경우 Water Level Exceeded! 문자열을 msg\_q\_alarm에 전송한다.
- TaskAlarm에선 메시지를 수신하고 콘솔 로그 메시지 패널 및 내부 로그 버퍼에 알람을 기록한다.

## 4. 제안서 대비 변경사항

프로젝트를 설계하는 과정에서 작성한 제안서와 실제 구현 결과 간에는 몇 가지 차이점이 발생하였다. 주요 변경 사항은 다음과 같다.

- Kernel Function 구성 변경  
제안서에서는 사용자 정의 Kernel Function으로 LogSystemEvent(), TriggerAlarm() 등을 명시하였지만 실제 구현에서는 일부 기능이 다른 Kernel Function으로 병합되거나 생략되었다.
  - LogSystemEvent는 기능을 분산하여 AddLog와 PrintLogPanel로 나누어 구현하였다.
  - TriggerAlarm는 기능을 단순화하여 경고 발생 시 이를 해당 Task에서 Message queue를 통해 TaskAlarm으로 전달하도록 구현하였다.
  - CheckComponentStatus 탬의 구성 요소 구조체를 기반으로 터빈, 게이트 등 주요 구성 요소의 손상도를 점검하고 상태 이상 발생 시 경고 발생을 유도하는 구조였으나 실제 구현에서는 핵심 기능에 집중하기 위해 생략하였다.
  - ApplyManualOverride는 TaskUserInput이 제외됨에 따라 생략되었다.
- TaskUserInput 제외  
제안서에서는 사용자의 입력을 처리하여 런타임 중 설정 값을 조작할 수 있는 TaskUserInput을 포함하였다. 실제로 이 Task의 구현은 일부 완료되었으나, 콘솔 UI 및 TaskTimeTicker와의 충돌로 인해 제외되었다.
- Task 구조 재조정  
제안서 상에서는 총 8개의 Task를 계획했으며 일부는 추상화 수준에서 기능이 중첩되어 있었다. 최종 구현에서는 다음과 같이 반영되었다.

| 제안서 명시 Task              | 실제 구현 Task             |
|--------------------------|------------------------|
| User Input Task (X)      | 제외됨 (TaskUserInput)    |
| Climate Simulation Task  | TaskReservoir          |
| Water Level Sensor Task  | TaskWaterSensor        |
| Gate Control Task        | TaskGateControl        |
| Turbine Control Task (X) | TaskGateControl로 기능 병합 |
| Alarm Task               | TaskAlarm              |
| System Monitoring Task   | TaskPrintStatus        |
| Logging Task             | TaskLog                |

Turbine Control Task의 로직은 TaskGateControl에 일부 구현되었고 필요 없다 생각한 내용은 생략하여 최종 결과물에 반영하였다.

## 5. 동작 예시

- config.txt 설정

 config.txt - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

```
RESERVOIR_AREA=10000
WATER_HEIGHT=80
INFLOW_FIXED=4000
INFLOW_VARIABLE=0
GATE_OPEN_PERCENT=30
LEVEL_LOW=60
LEVEL_MID=100
LEVEL_HIGH=130
THRESHOLD_HEIGHT=150
RAIN_MIN=100
RAIN_MAX=500
STORM_MIN=1000
STORM_MAX=3000
```

시뮬레이터는 실행 시 config.txt 파일을 읽어 초기 상태를 설정한다. 이 파일은 댐의 기본 환경 설정값과 제어 기준값을 포함하고 있으며 다음 항목으로 구성된다.

| 항목                    | 설명                                             |
|-----------------------|------------------------------------------------|
| RESERVOIR_AREA        | 댐 저수지의 수면 면적 (m <sup>2</sup> 단위)<br>수위 계산에 사용됨 |
| WATER_HEIGHT          | 시뮬레이션 시작 시 초기 수위 (m 단위)                        |
| INFLOW_FIXED          | 고정 유입량 (m <sup>3</sup> /s 단위)                  |
| INFLOW_VARIABLE       | 초기 가변 유입량 (기후 상태에 따라 변경)                       |
| GATE_OPEN_PERCENT     | 초기 수문 개방률 (% 단위)                               |
| LEVEL_LOW             | 수문 제어를 위한 수위 기준 (낮음)<br>자동 제어에서 10% 개방 수준      |
| LEVEL_MID             | 수문 제어를 위한 수위 기준 (중간)<br>자동 제어에서 30% 개방 수준      |
| LEVEL_HIGH            | 수문 제어를 위한 수위 기준 (높음)<br>자동 제어에서 60% 개방 기준      |
| THRESHOLD_HEIGHT      | 수위 초과 알람 발생 임계치 (m 단위)                         |
| RAIN_MIN / RAIN_MAX   | 우천 기후에서 가변 유입량 범위                              |
| STORM_MIN / STORM_MAX | 폭우 기후에서 가변 유입량 범위<br>(보통 RAIN보다 더 높게 설정)       |

## - main.exe

C:\Users\User\OneDrive - 광운대학교\임베디드시스템SW설계\project\PROJECT\main.exe

```
Time: 00:02:47:500 [LOG MESSAGES]
===== DAM STATUS MONITOR =====
Elapsed Time      : 169 sec [GATE] 30% -> 60%
Water Height     : 100.26 m [GATE] 60% -> 30%
Fixed Inflow     : 4000.00 m^3/s [GATE] 30% -> 60%
Variable Inflow  : 2248.00 m^3/s [CLIMATE] Clear -> Storm
Gate Open Rate   : 60 % [GATE] 60% -> 30%
Outflow Rate     : 6600.00 m^3/s [GATE] 30% -> 60%
Power Output     : 0.08 kWh [GATE] 60% -> 30%
Total Power      : 7.67 kWh [GATE] 30% -> 60%
Total Outflow    : 607200.00 m^3 [GATE] 60% -> 30%
Alarm Count      : 0 [CLIMATE] Storm -> Clear
Manual Override  : NO
```

프로젝트 실행 파일인 main.exe를 실행하면, 콘솔 창에 실시간으로 댐의 상태와 로그 메시지가 출력된다. 위 화면은 시스템이 정상적으로 동작할 때의 모습을 보여주며, 댐의 정보와 함께 수위 변화, 수문 개방률 변화, 알람 발생 여부 등을 관찰할 수 있다.

C:\Users\User\OneDrive - 광운대학교\임베디드시스템SW설계\project\PROJECT\main.exe

```
Time: 00:00:30:100 [LOG MESSAGES]
===== DAM STATUS MONITOR =====
Elapsed Time      : 31 sec Water Level Exceeded!
Water Height     : 152.21 m [CLIMATE] Clear -> Storm
Fixed Inflow     : 30000.00 m^3/s Water Level Exceeded!
Variable Inflow  : 1355.00 m^3/s [GATE] 60% -> 90%
Gate Open Rate   : 90 % [GATE] 30% -> 60%
Outflow Rate     : 9900.00 m^3/s
Power Output     : 0.12 kWh
Total Power      : 2.75 kWh
Total Outflow    : 217800.00 m^3
Alarm Count      : 2
Manual Override  : NO
```

위 화면은 경고 발생 시 동작 예시이다. 수위가 임계값인 150m를 초과했을 때 알람 카운트가 증가하고, 경고 메시지가 우측 로그 메시지 패널에 출력되는 것을 관찰할 수 있다. TaskWaterSensor가 수위를 감지하고, TaskAlarm이 경고 메시지를 수신한 후 로그를 표시하여 TaskGateControl이 자동으로 대응하는 모습을 통해 Task 간 협업 구조가 정상적으로 동작하고 있음을 보여준다.

- status.txt

 status.txt - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

```
Time: 00:03:28
WaterHeight: 100.06 m
InflowFixed: 4000.00 m^3/s
InflowVariable: 380.00 m^3/s
GateOpenRate: 60 %
OutflowRate: 6600.00 m^3/s
PowerOutput: 0.08 kWh/s
TotalPower: 10.08 kWh
TotalOutflow: 798600.00 m^3
AlarmCount: 0
ManualOverride: NO
```

```
-----
[DAM STATUS]
Time: 00:03:38
WaterHeight: 99.82 m
InflowFixed: 4000.00 m^3/s
InflowVariable: 1305.00 m^3/s
GateOpenRate: 30 %
OutflowRate: 3300.00 m^3/s
PowerOutput: 0.04 kWh/s
TotalPower: 10.63 kWh
TotalOutflow: 841500.00 m^3
AlarmCount: 0
ManualOverride: NO
```

```
-----
[DAM STATUS]
Time: 00:03:48
WaterHeight: 100.17 m
InflowFixed: 4000.00 m^3/s
InflowVariable: 2184.00 m^3/s
GateOpenRate: 60 %
OutflowRate: 6600.00 m^3/s
PowerOutput: 0.08 kWh/s
TotalPower: 11.29 kWh
TotalOutflow: 894300.00 m^3
AlarmCount: 0
ManualOverride: NO
```

status.txt 파일은 TaskLog에 의해 10초 주기로 댐의 현재 상태를 기록하는 로그 파일이다. 이 파일은 시뮬레이션이 진행되는 동안 지속적으로 업데이트되며, 각 시점의 댐 상태를 텍스트 기반으로 저장한다.



- message.txt

messages.txt - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

```
[00:00:00] [CLIMATE] Clear -> Storm
[00:00:00] [CLIMATE] Clear -> Storm
[00:00:19] [CLIMATE] Storm -> Rain
[00:00:29] [CLIMATE] Rain -> Clear
[00:00:39] [CLIMATE] Clear -> Storm
[00:00:49] [CLIMATE] Storm -> Rain
[00:00:59] [CLIMATE] Rain -> Clear
[00:01:09] [CLIMATE] Clear -> Storm
[00:01:19] [CLIMATE] Storm -> Clear
[00:01:29] [CLIMATE] Clear -> Rain
[00:01:59] [CLIMATE] Rain -> Clear
[00:02:09] [CLIMATE] Clear -> Rain
[00:02:19] [CLIMATE] Rain -> Storm
[00:02:22] [GATE] 30% -> 60%
[00:02:23] [GATE] 60% -> 30%
[00:02:24] [GATE] 30% -> 60%
[00:02:28] [CLIMATE] Storm -> Clear
[00:02:29] [GATE] 60% -> 30%
[00:02:31] [GATE] 30% -> 60%
[00:02:32] [GATE] 60% -> 30%
[00:02:35] [GATE] 30% -> 60%
[00:02:36] [GATE] 60% -> 30%
[00:02:38] [CLIMATE] Clear -> Storm
```

message.txt 파일은 시스템에서 발생한 주요 메시지를 시간 순으로 기록한 텍스트 로그 파일이다. AddLog()를 통해 TaskAlarm 혹은 다른 Task들에서 출력된 로그가 저장된다.